DP Computer Science: B 2.3.4 Functions and Modularisation

---

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Define the concept of **functions** and their role in **modularisation**
- Define and call **functions** with different inputs and return values
- Explain the difference between **local and global scope**
- Apply **modularisation principles** to create well-structured and maintainable code

---

Key Vocabulary

Term	Definition
Function	A reusable block of code that performs a specific task
Modularisation	Breaking down a program into smaller, independent modules (functions)
Parameter	A variable passed into a function to provide input
Argument	The actual value passed to a function when called
Return value	The value a function sends back to the caller
Local variable	A variable declared inside a function; only accessible within that function
Global variable	A variable declared outside any function; accessible throughout the program
Scope	The region of a program where a variable is accessible

IB ATL Skills

Skill	Description
Thinking	Critical thinking (analysing and evaluating issues); creative thinking (generating novel ideas)
Communication	Exchanging thoughts and information effectively through interaction
Self-Management	Managing time and tasks effectively
Research	Finding, interpreting, judging, and creating information
Collaboration	Working effectively with others

---

2. Introduction to Functions

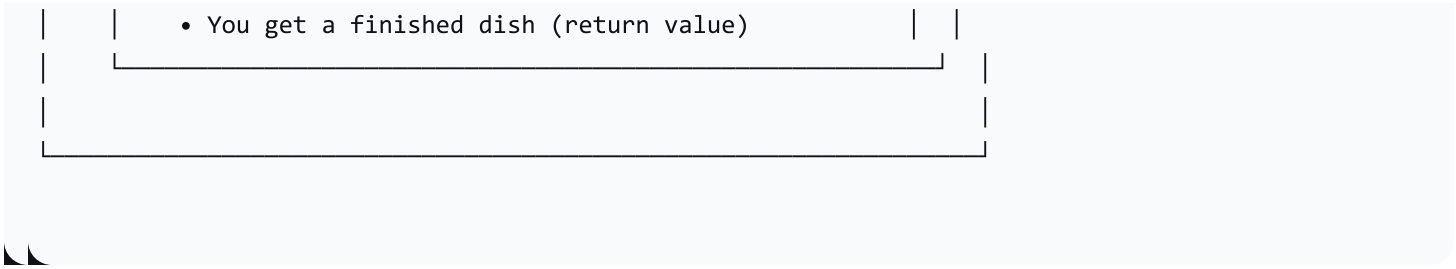
What are Functions?

> **Functions are reusable blocks of code that perform specific tasks. Think of them as mini-programs within a program.**

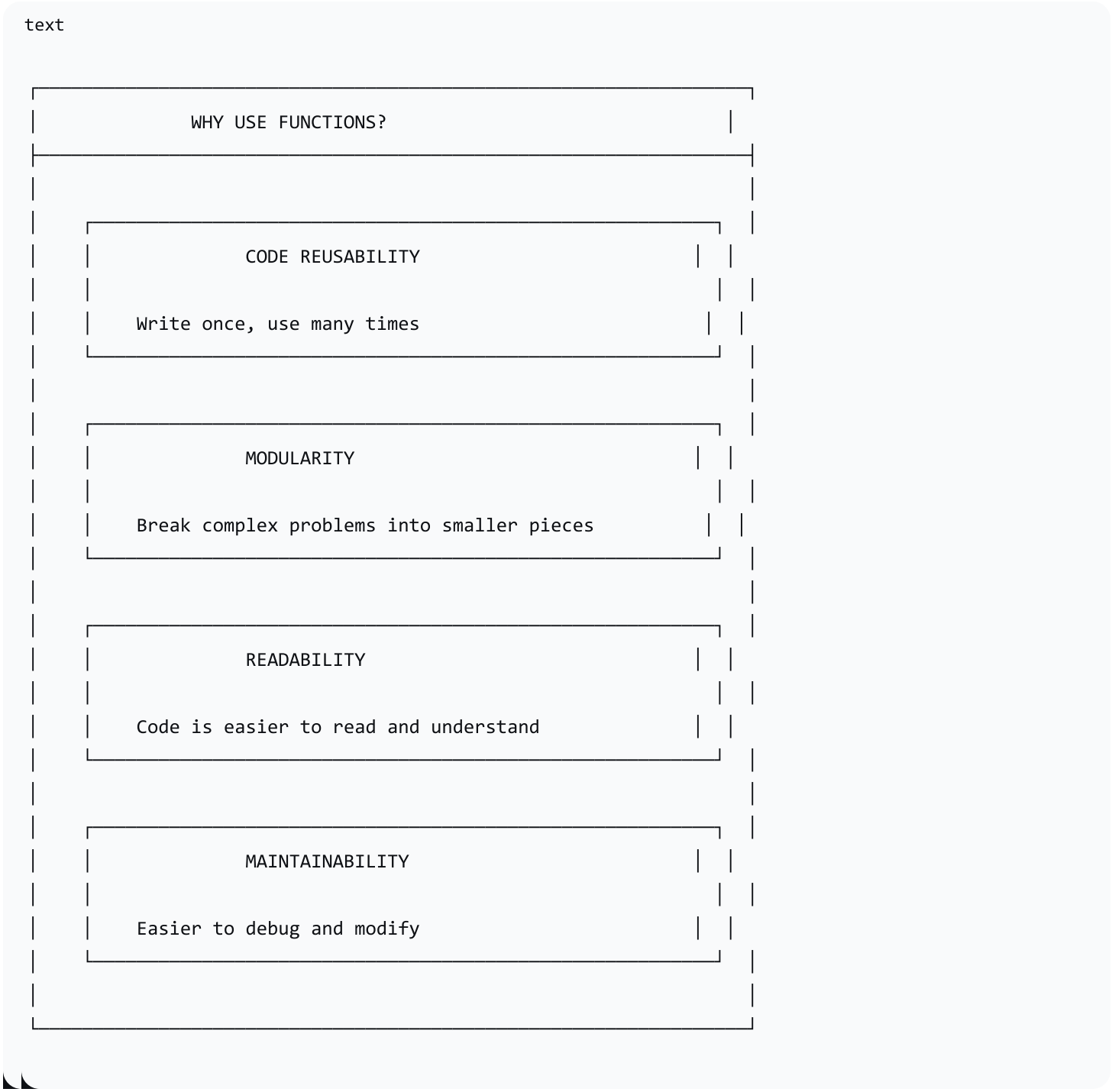
The Analogy

```
text

┌─────────────────────────────────────────────────────────────┐
│              FUNCTION ANALOGY                               │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│    ┌─────────────────────────────────────────────────┐      │
│    │              CHEF IN A KITCHEN                  │  │
│    │                                                 │  │
│    │    • You don't need to know how every dish is made  │  │
│    │    • You just need to know the recipe (function)    │  │
│    │    • You provide ingredients (parameters)           │  │
```



Why Use Functions?



3. Defining and Calling Functions

Function Structure

python

```python
# Python
def function_name(parameters):
    """Docstring (optional)"""
    # Function body
    return value  # Optional
```

java

```java
// Java
public static returnType functionName(parameters) {
    // Function body
    return value;  // Required if returnType is not void
}
```

Example: Calculating Area

python

```python
# Python
def calculate_area(length, width):
    """Calculate the area of a rectangle"""
    area = length * width
    return area

# Calling the function
result = calculate_area(5, 3)
print(f"Area: {result}")  # Output: Area: 15
```

java

```java
// Java
public static double calculateArea(double length, double width) {
    double area = length * width;
```

```
    return area;
}

// Calling the function
double result = calculateArea(5, 3);
System.out.println("Area: " + result);  // Output: Area: 15.0
```

Function Components

Component	Description	Example
Function Name	Identifies the function	`calculate_area`
Parameters	Inputs to the function	`length`, `width`
Body	Code that performs the task	`area = length * width`
Return Value	Output from the function	`area`
Call	Using the function	`calculate_area(5, 3)`

Multiple Parameters and Return Values

python

```
# Python
def calculate_rectangle(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter

# Calling with multiple return values
area, perimeter = calculate_rectangle(5, 3)
print(f"Area: {area}, Perimeter: {perimeter}")
```

java

```java
// Java
public static double[] calculateRectangle(double length, double width) {
    double area = length * width;
    double perimeter = 2 * (length + width);
    return new double[]{area, perimeter};
}

// Calling
double[] result = calculateRectangle(5, 3);
System.out.println("Area: " + result[0] + ", Perimeter: " + result[1]);
```

---

4. Parameters vs. Arguments

Parameters (Definition)

> Parameters are the variables listed in a function's definition. They act as placeholders for the input values.

Arguments (Call)

> Arguments are the actual values passed to a function when it is called.

```python
# Python: Parameters: a, b
def add(a, b):
    return a + b

# Arguments: 5, 3
result = add(5, 3)  # result = 8
```

Positional vs. Keyword Arguments

```python
# Python: Positional arguments (order matters)
result = add(5, 3)  # a=5, b=3
```

```python
# Python: Keyword arguments (order doesn't matter)
result = add(b=3, a=5)  # a=5, b=3
```

Default Parameters

python

```python
# Python: Default values
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alice")        # Hello, Alice!
greet("Bob", "Hi")    # Hi, Bob!
```

java

```java
// Java: Method overloading (no default parameters)
public static void greet(String name) {
    System.out.println("Hello, " + name + "!");
}

public static void greet(String name, String greeting) {
    System.out.println(greeting + ", " + name + "!");
}
```

---

5. Scope: Local vs. Global

What is Scope?

> Scope defines the region of a program where a variable is accessible.

Local Variables

> Local variables are declared inside a function and are only accessible within that function.

```python
# Python
def my_function():
    x = 10  # Local variable
    print(x)

my_function()  # Output: 10
print(x)  # ❌ Error: x is not defined outside the function
```

Global Variables

> Global variables are declared outside any function and are accessible throughout the program.

```python
# Python
x = 10  # Global variable

def my_function():
    print(x)  # ✅ Accessible

my_function()  # Output: 10
print(x)  # Output: 10
```

Modifying Global Variables

```python
# Python: Modifying a global variable
x = 10

def change_value():
    global x  # Declare intent to modify global variable
    x = 20

change_value()
print(x)  # Output: 20
```

```java
// Java: Modifying a global (static) variable
public class Example {
    static int x = 10;

    public static void changeValue() {
        x = 20;  // Modifies global variable
    }

    public static void main(String[] args) {
        changeValue();
        System.out.println(x);  // Output: 20
    }
}
```

Scope Comparison

Feature	Local Variable	Global Variable
Declaration	Inside a function	Outside any function
Accessibility	Only within that function	Throughout the program
Lifetime	Created when function is called; destroyed when it returns	Created when program starts; destroyed when it ends
Scope	Function scope	Global scope
Analogy	A sticky note used for one task	A public whiteboard

---

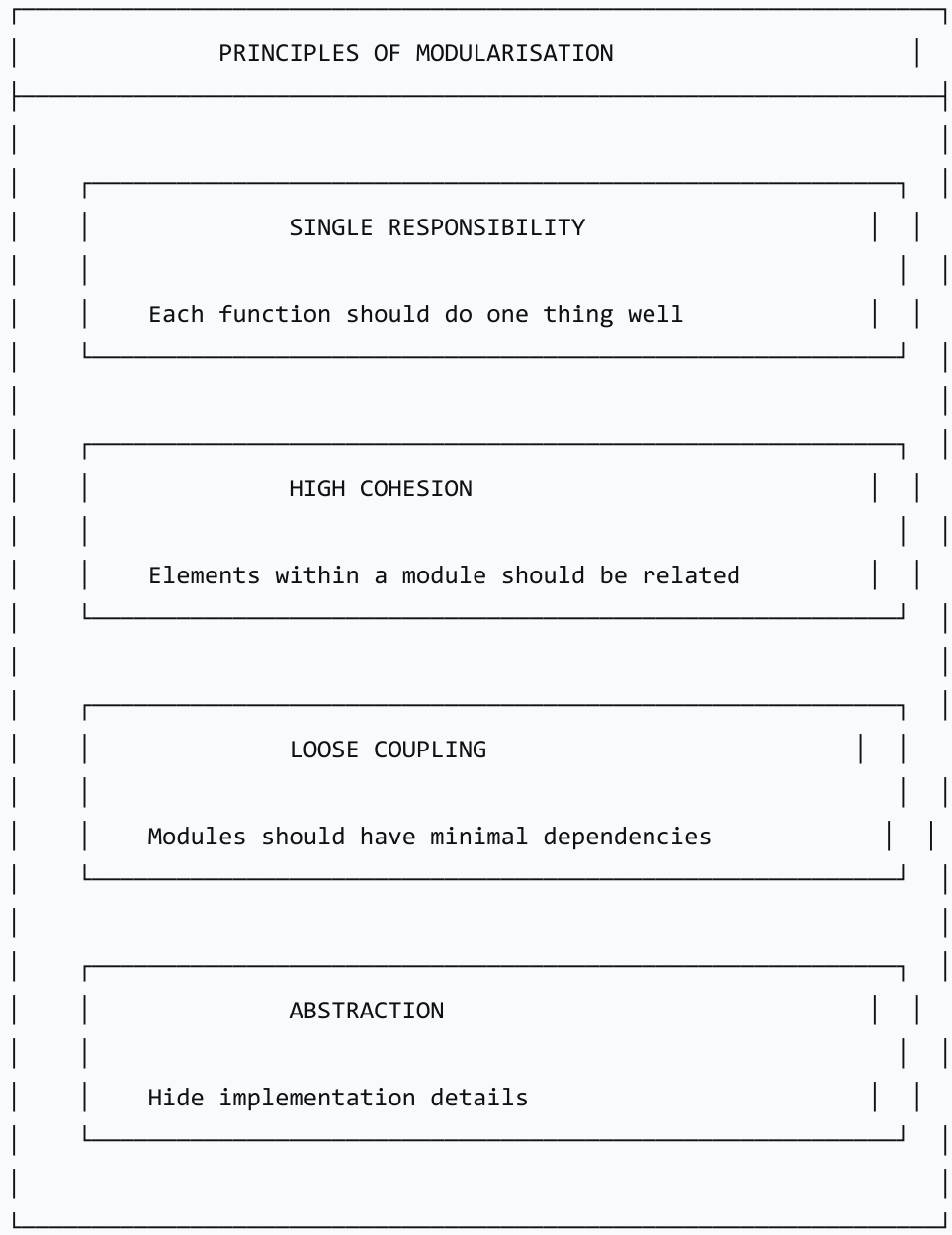
6. Modularisation

What is Modularisation?

> **Modularisation is the process of breaking down a program into smaller, independent modules (functions) that each perform a specific task.**

Principles of Modularisation

text



Modularisation Example

python

```python
# Python: Without modularisation (messy)
def main():
    # Student management
    students = ["Alice", "Bob", "Charlie"]
```

```
    for student in students:
        print(f"Processing {student}...")

    # Course management
    courses = ["Math", "Science", "English"]
    for course in courses:
        print(f"Processing {course}...")

    # Teacher management
    teachers = ["Mr. A", "Ms. B", "Dr. C"]
    for teacher in teachers:
        print(f"Processing {teacher}...")
```

python

```python
# Python: With modularisation (clean)
def process_students(students):
    for student in students:
        print(f"Processing {student}...")

def process_courses(courses):
    for course in courses:
        print(f"Processing {course}...")

def process_teachers(teachers):
    for teacher in teachers:
        print(f"Processing {teacher}...")

def main():
    students = ["Alice", "Bob", "Charlie"]
    courses = ["Math", "Science", "English"]
    teachers = ["Mr. A", "Ms. B", "Dr. C"]

    process_students(students)
    process_courses(courses)
    process_teachers(teachers)
```

---

7. Worksheet Activities

Activity 1: Function Definition and Calling

Task: Complete the function definitions and call them correctly.

```python

# 1. Define a function called "square" that takes a number and returns its square
# 2. Define a function called "is_even" that takes a number and returns True if even
# 3. Define a function called "sum_list" that takes a list and returns the sum
# 4. Call each function and print the results
```

Activity 2: Modularising a Program

Task: Refactor this program by creating functions.

```python

# Refactor this program:
# 1. Calculate the sum of numbers from 1 to 10
# 2. Calculate the sum of numbers from 1 to 20
# 3. Calculate the sum of numbers from 1 to 50
```

Solution:

```python

def calculate_sum(start, end):
    """Calculate the sum of numbers from start to end"""
    total = 0
    for i in range(start, end + 1):
        total += i
    return total

# Usage
print(calculate_sum(1, 10))  # 55
print(calculate_sum(1, 20))  # 210
print(calculate_sum(1, 50))  # 1275
```

Activity 3: Temperature Converter

Task: Write functions to convert between Celsius and Fahrenheit.

```python
def celsius_to_fahrenheit(celsius):
    # Your code here
    pass

def fahrenheit_to_celsius(fahrenheit):
    # Your code here
    pass

# Test the functions
print(celsius_to_fahrenheit(100))  # 212
print(fahrenheit_to_celsius(212))  # 100
```

---

8. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	10 min	What are functions? Why use them?
Defining/Calling	15 min	Function structure; examples; Activity 1
Modularisation/Scope	20 min	Modularisation; local vs. global; Activity 2
Conclusion	10 min	Key takeaways; further exploration

---

9. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions

Assessment Type	Description
Worksheet	Evaluate understanding of functions and modularisation
Coding Challenges	Assign projects using functions

10. Differentiation

Level	Strategy
Support	Provide scaffolded examples and function templates
Challenge	Explore recursion, higher-order functions

11. Resources

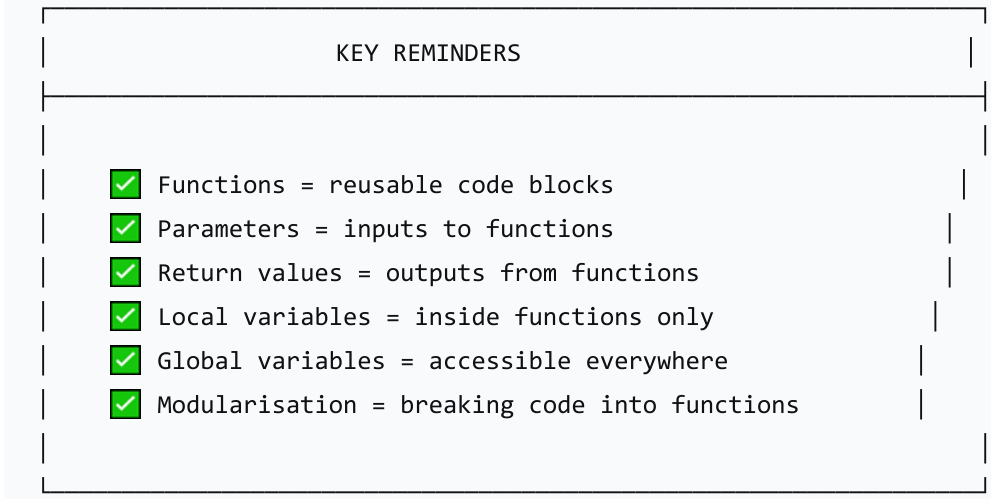
- **Presentation** (Slide Deck)
- **eBook** (B2.3.4 with examples)
- **Worksheets:** Functions and modularisation exercises
- **Worksheet Answers**

12. Summary: Key Takeaways

Concept	Key Point
Function	Reusable block of code
Parameter	Input to a function
Return Value	Output from a function
Local Variable	Inside function only

Concept	Key Point
Global Variable	Throughout program
Modularisation	Breaking code into functions

```
text
```



> *"Functions are the building blocks of well-structured programs—they promote reusability, readability, and maintainability."*